

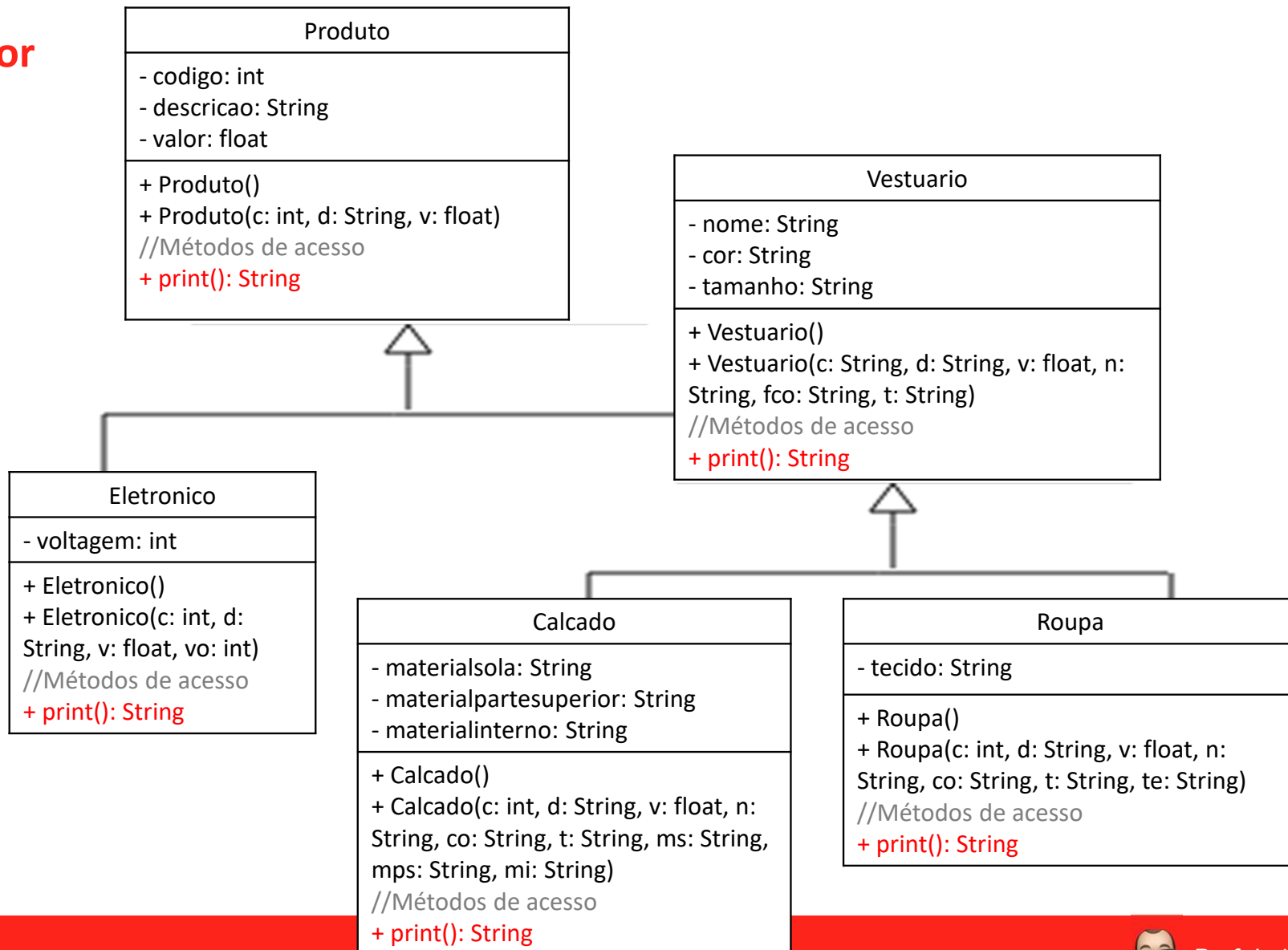
Desenvolvimento Back-End



Aula 07

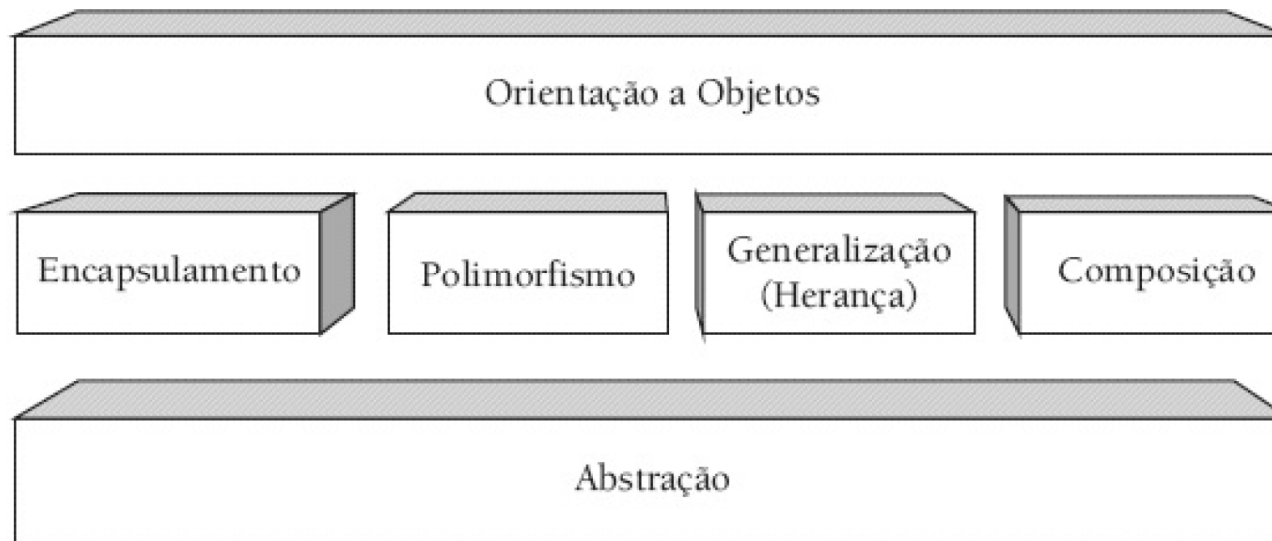
- Classes Abstratas
- Interface
- Composição

Aula anterior



Abstração

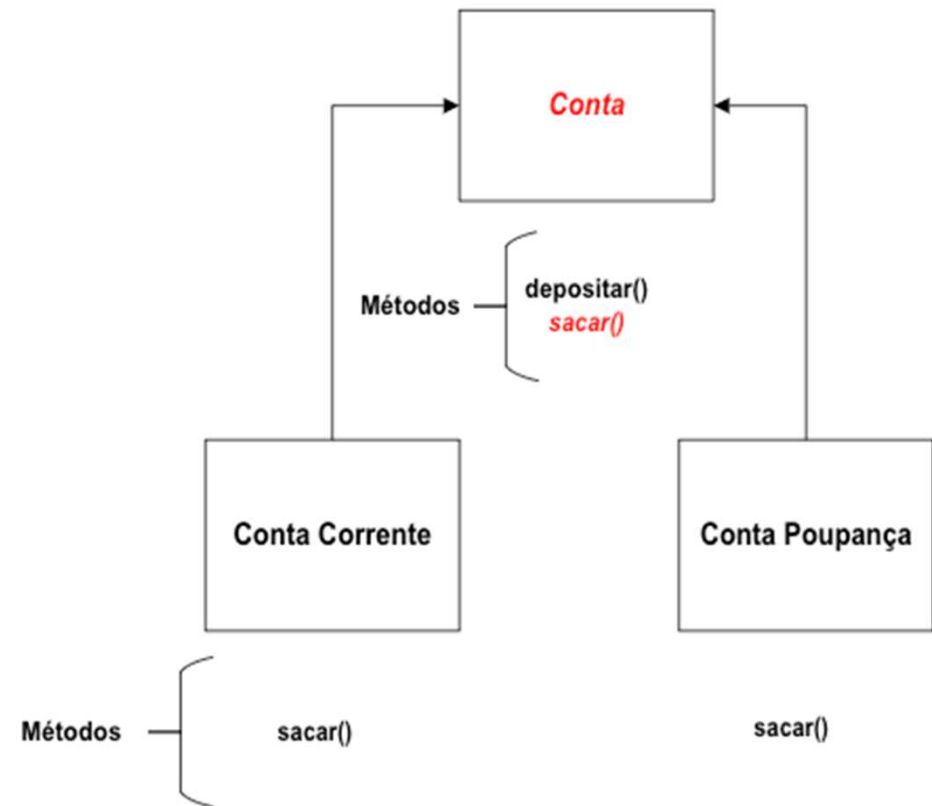
A partir desse momento serão apresentados **outros conceitos** do paradigma da orientação a objetos. Todos esses conceitos são, na verdade, a **aplicação** de um único **conceito** mais básico, o **princípio da abstração**. Como vimos, a abstração de algo é dependente da **perspectiva** (contexto) sobre a qual uma coisa é analisada: **o que é importante em um contexto pode não ser importante em outro**.



Classe Abstrata

Uma classe abstrata é uma classe que:

- Provê organização;
- **Não possui “instâncias”;**
- Pode ter métodos abstratos ou concretos;
- Um método abstrato herdado por uma classe filha, deve ser implementado pela classe filha, desde que a mesma seja concreta;
- Uma classe concreta não pode ter métodos abstratos.



Classe Abstrata

Os métodos abstratos definidos em uma classe abstrata devem obrigatoriamente implementados em uma classe concreta. Mas se uma classe abstrata herdar outra classe abstrata, a classe que herda não precisa implementar os métodos abstratos.

Indica que todas as classes filhas concretas devem reescrever esse método e ele deve ser um método concreto nas classes filhas;

Os métodos que são abstratos têm um comportamento diferente nas classes filhas, por isso não possuem corpo;

A assinatura do método abstrato é da seguinte forma:

`public abstract tipoRetorno NomeDoMetodo();`

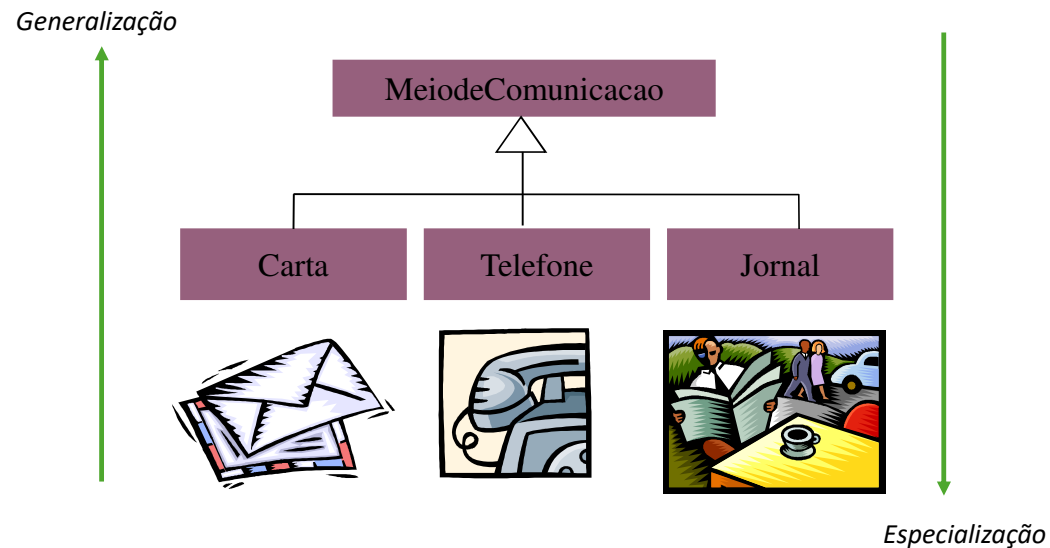
Palavra-chave que identifica o método como abstrato

Não é definida uma forma de implementação específica.

Abstração de Dados – Classes Abstratas

Exemplo

Nos ajuda a lidar com a complexidade.



A classe *MeiodeComunicacao* neste caso é abstrata e pode representar um domínio.

Classes Abstratas

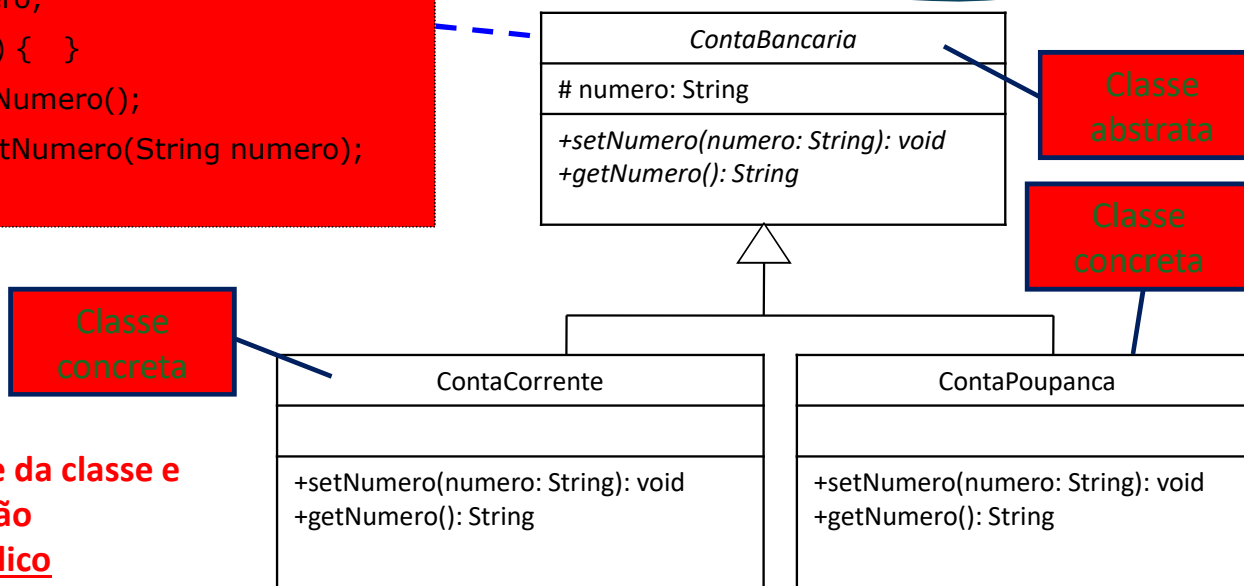
Uma classe abstrata é uma classe que:

- Provê organização
- **Não possui “instâncias”**
- Pode ter métodos abstratos ou concretos

```
public abstract class ContaBancaria {  
    protected String numero;  
    public ContaBancaria() { }  
    public abstract int getNumero();  
    public abstract void setNumero(String numero);  
}
```

Se você definir um método abstrato sua classe **OBRIGATORIAMENTE**

Você pode ter métodos concretos em uma classe abstrata



No diagrama o nome da classe e métodos abstratos são apresentados em itálico

Classes Abstratas

- São criadas quando não se pretende criar objetos a partir delas
- São normalmente usadas como superclasses
- Se caracterizam por serem muito genéricas
- A assinatura da classe abstrata é da seguinte forma:

```
public abstract class NomeDaClasse{
```

```
...
```

```
}
```

Palavra-chave que identifica a classe
como abstrata

Métodos Abstratos

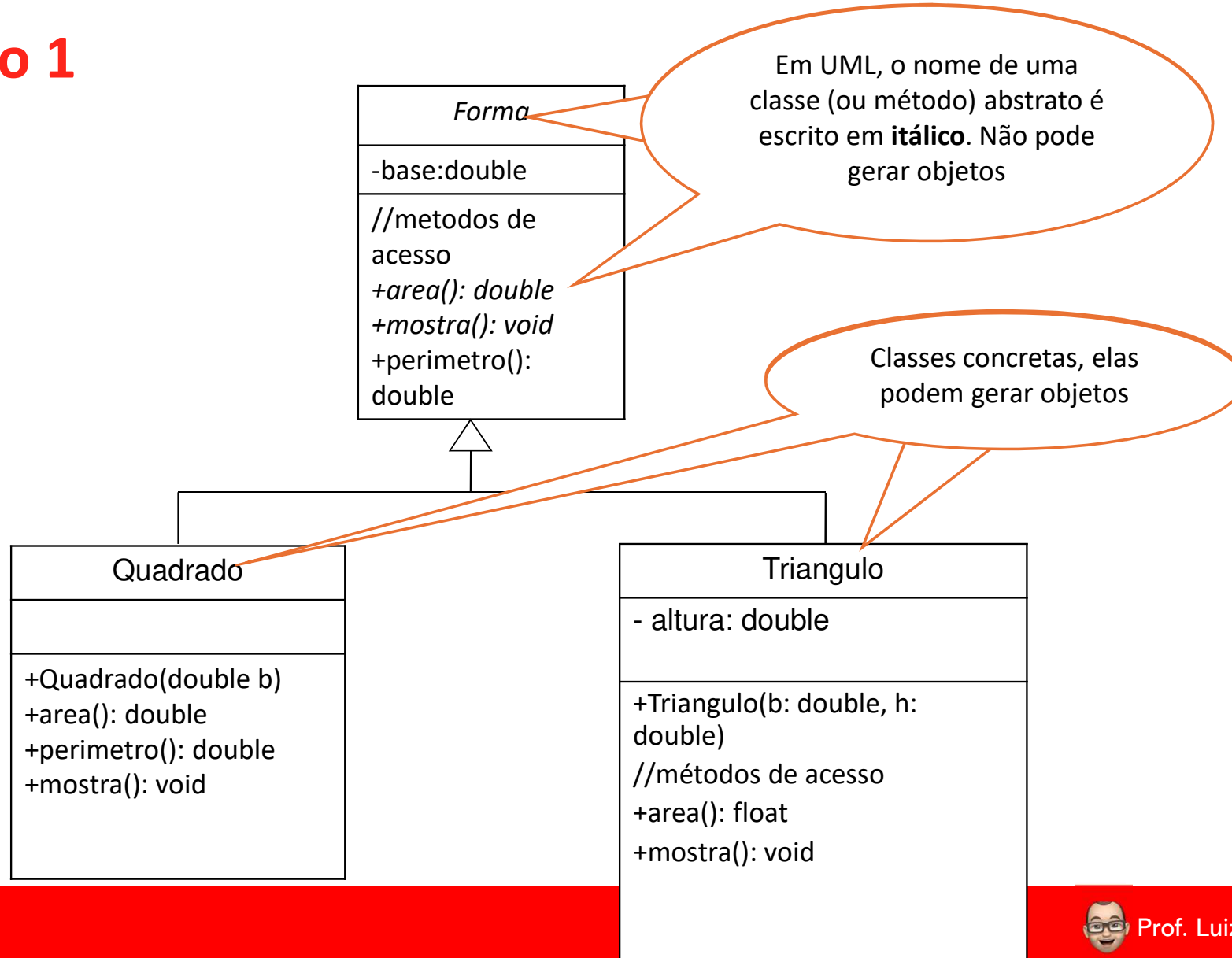
- Os métodos abstratos definidos em uma classe abstrata devem obrigatoriamente implementados em uma classe concreta. Mas se uma classe abstrata herdar outra classe abstrata, a classe que herda não precisa implementar os métodos abstratos.
- Indica que todas as classes filhas concretas devem reescrever esse método e ele deve ser um método concreto nas classes filhas;
- Os métodos que são abstratos têm um comportamento diferente nas classes filhas, por isso não possuem corpo;
- A assinatura do método abstrato é da seguinte forma:

`public abstract tipoRetorno NomeDoMetodo();`

Palavra-chave que identifica o método como abstrato

Não é definida uma forma de implementação específica.

Exemplo 1

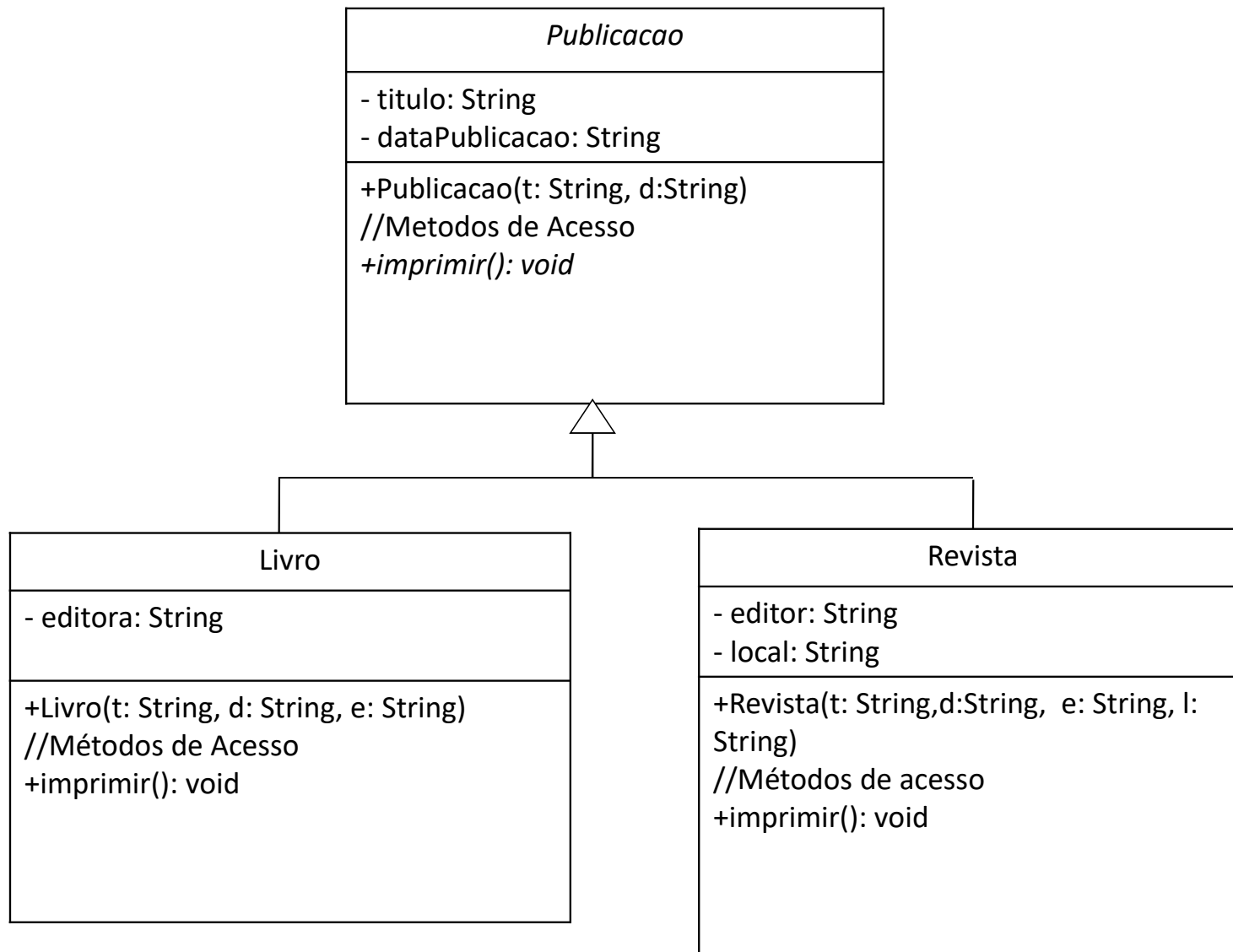


Exemplo 1

```
public abstract class Forma {  
    //Atributos  
  
    //Metodos  
    public abstract float area();  
    public abstract void mostra();  
  
    public float perimetro(){ return 0f; }  
}
```

Métodos abstratos, não possuem implementação

Exercício 01



Interface

- São estruturas similares às classes abstratas, que definem a especificação de funcionalidades, sem a implementação das mesmas.
- Uma interface Java pode ser definida como uma “**classe abstrata pura**”, pois não pode possuir atributos (exceto constantes) nem definições de métodos, nem construtor.

Interface

- Apenas métodos públicos podem ser declarados nelas, mas não podem ser definidos.
 - ✓ Todos os métodos declarados dentro de uma interface são, implicitamente, *public* e *abstract*
- Da mesma forma, não é possível definir atributos - apenas constantes públicas.
 - ✓ Todos os atributos declarados dentro de uma interface são, implicitamente, *public*, *static* e *final*
- São proibidas algumas declarações dentro das interfaces, dentre elas:
 - ✓ *private*, *protected*, *transient*, *volatile* e *synchronized*.

Interface

- A palavra-chave *interface* é utilizada em vez de *class* no cabeçalho da declaração:

```
public interface nomeInterface { ... }
```

- Todos os métodos são abstratos. A palavra-chave *abstract* não é necessária

Classes Abstratas e Interfaces

- A diferença entre uma classe abstrata e uma interface Java é que a **interface obrigatoriamente não pode ter um “corpo” associado.**
- Enquanto uma **classe abstrata** é “**estendida**” (palavra chave *extends*) por classes derivadas, uma **interface** Java é “**implementada**” (palavra chave *implements*) por outras classes.

Classes Abstratas e Interfaces

```
public class nomeClasse implements nomeInterface
```

Ou

```
public class nomeClasse implements nomeInterface1, nomeInterface2
```

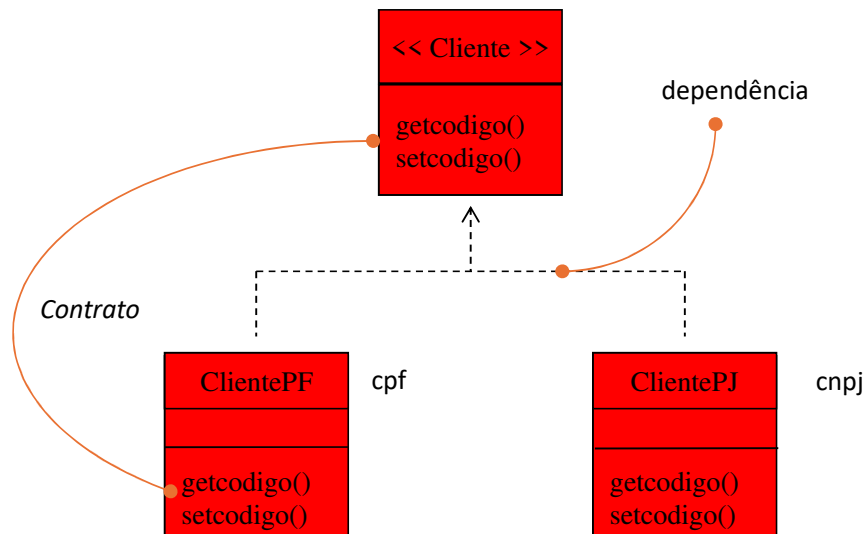
Ou

```
public class nomeClasse extends nomeHeranca implements  
    nomeInterface
```

A interface **não faz parte da hierarquia** de classes em Java.

Implementação de Interfaces

- Uma interface estabelece uma espécie de contrato que é obedecido por uma classe.
 - ✓ Quando uma classe implementa uma interface, garante-se que todas as funcionalidades especificadas pela interface serão oferecidas pela classe.



- Uma **classe pode implementar várias interfaces**, sendo um mecanismo elegante para se trabalhar com herança múltipla em Java.

Interfaces - Vantagens

- Implementar similaridades entre classes não-relacionadas, sem forçar relacionamentos na hierarquia de classes.
- Definir métodos que uma ou mais classes devam implementar.
- Revelar apenas as interfaces de um objeto, sem revelar a estrutura de sua classe, ajuda assim esconder a complexidade da arquitetura de componentes;
- Oferece uma forma simplificada de implementar herança múltipla.
- Facilitar o entendimento do código – separação entre o que a classe “é” do que ela “faz”.

Interfaces – Exemplo 2

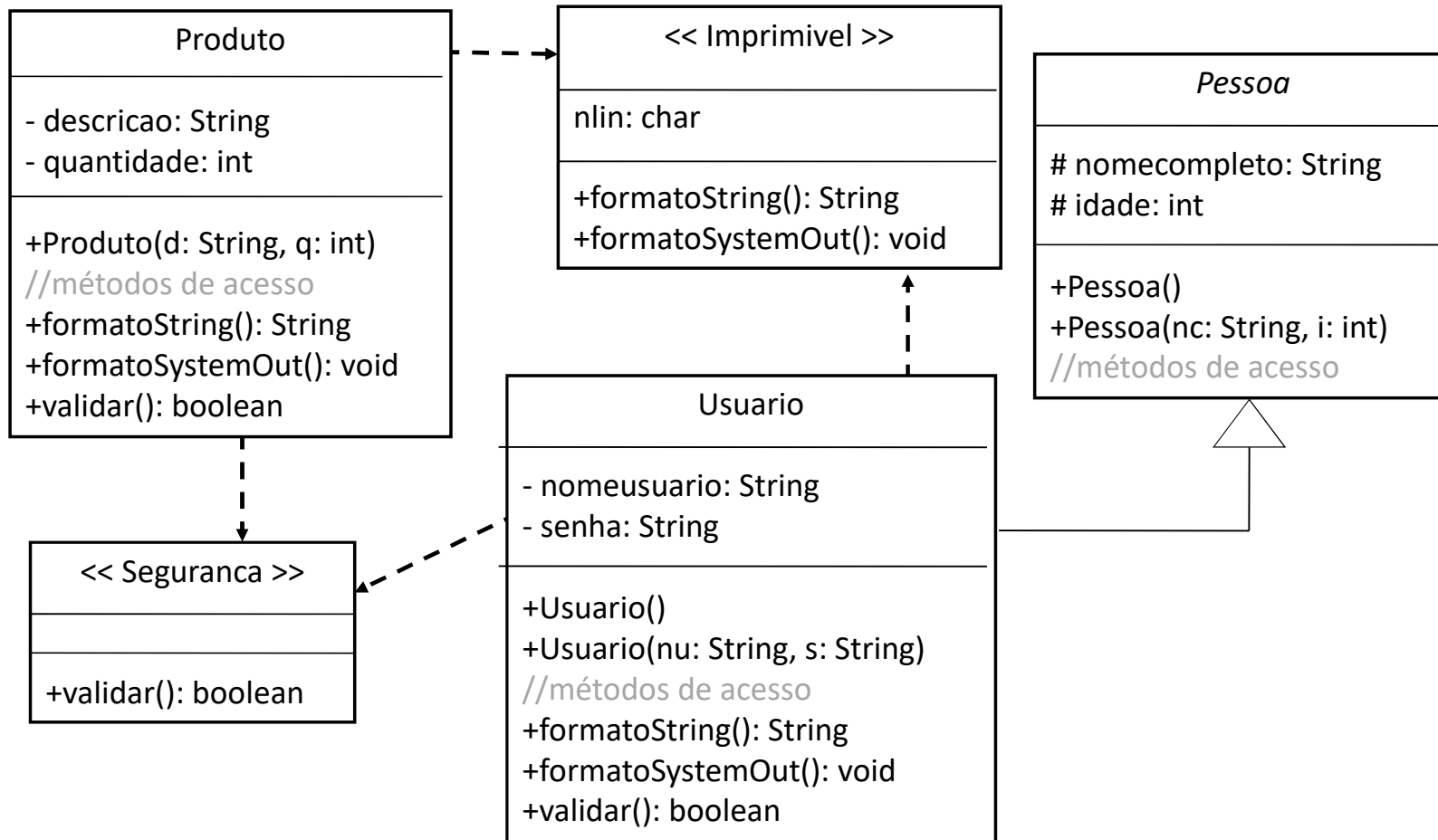
```
public interface Imprimivel {  
    //constante  
    final char nlin='\n';  
  
    //métodos  
    public String formatoString();  
    public void formatoSystemOut();  
}
```

<< Imprimivel >>
nlin: char
+formatoString(): String +formatoSystemOut(): void

```
public interface Seguranca {  
    public boolean validar();  
}
```

<< Seguranca >>
+validar(): boolean

Exemplo 2



Interfaces - Exemplo

- Todas as classes que implementem a interface “Imprimivel” terão a “**obrigação**” de implementar os métodos `formatoString()` e `formatoSystemOut()`;
- Isso faz com que você tenha um padrão, e não importa como os métodos serão implementados, o usuário saberá que esses métodos existirão;
- Outras classes como `ContaCorrente`, `Alunos`, `FichaClinica`, etc, podem implementar essa interface e terão os métodos da interface também.

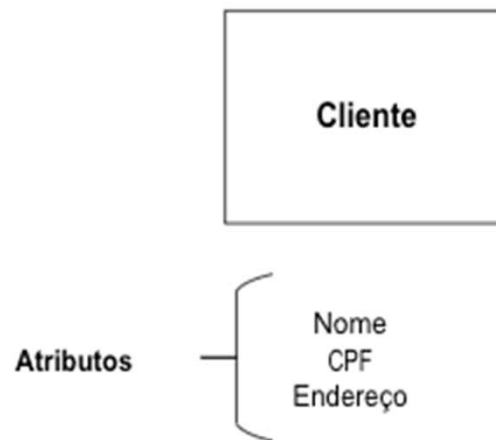
Interfaces x Classes Abstratas

- Use classes abstratas quando você quiser definir um “*template*” para subclasses e você possui alguma implementação (métodos concretos) que todas as subclasses podem utilizar.
- Use interfaces quando você quiser definir uma regra que todas as classes que implementem a interface devem seguir, independentemente se pertencem a alguma hierarquia de classes ou não.

Composição

É **natural** para nós pensar em coisas do mundo real como **objetos compostos** de **outros objetos**. Por exemplo, um livro é composto de páginas; páginas possuem parágrafos; parágrafos possuem frases e assim por diante.

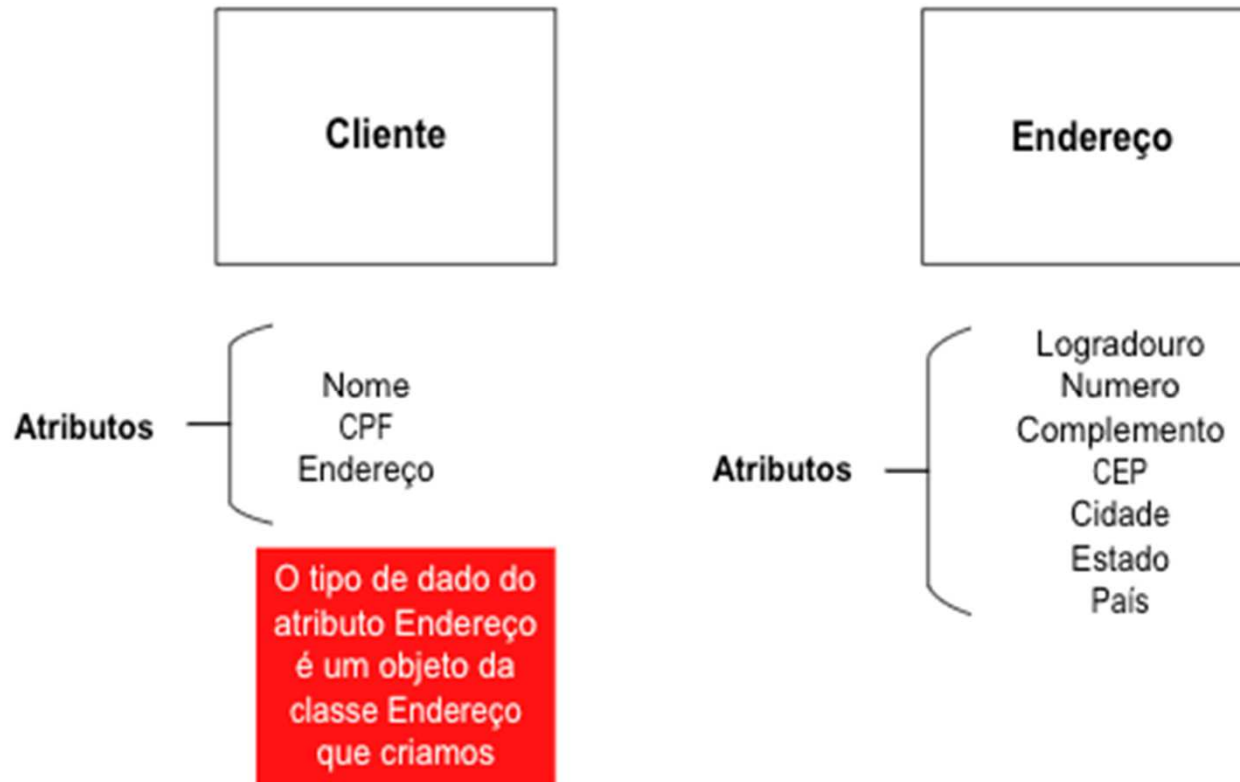
- A **composição** permite que criemos **objetos** a partir da **reunião** de outros **objetos**.



Vamos falar do endereço...
Rua, Avenida, Viela, Travessa,
Alameda...
Número...
Complemento...
CEP
Bairro
Cidade
Estado
País

O endereço é um atributo que podemos considerar multivalorado

Composição



Já sei tudo?

- Só isso? Já sei tudo que diz respeito à orientação a objetos?
- Não, ainda há um longo caminho pela frente, com muitos conceitos importantes ;
- Agora que tudo começa a ficar realmente divertido! 😁 😊 (é sério!)





That's all Folks!